

Approaches for UAV-Satellite Crossview Matching Task

I. Problem Background

该课题聚焦于“GNSS拒止环境下的无人机视觉定位”。其核心任务是通过匹配无人机拍摄的实景图像（Drone View）与预先存储的卫星遥感图像（Satellite View），来确定无人机的绝对地理位置。我计划将问题分成两个部分：一是粗检索，即在若干张预制的卫星遥感图像（Satellite View）中，找出最有可能包含目前无人机视角（Drone View）里的图像的那一张（或者最有可能的几张，看 Top-K 指标）；二是细匹配，针对挑出来的卫星图，要更精细的定位到无人机的准确位姿，用 6DoF 来表示。本次先把关注点放在 Task 1 也就是检索任务上。这一任务在学术界被称为 **Cross-View Geo-Localization**。相比于传统的图像检索，我总结了该任务面临着有挑战性的“视角鸿沟”：

- 1. 几何视角差异**：卫星图是垂直俯视（Orthogonal），而无人机图往往包含倾斜角度（Oblique）且存在任意的飞行旋转。这导致同一栋建筑在两张图中的几何形状可能会截然不同。
- 2. 环境干扰**：卫星图通常拍摄于理想条件下，而无人机实时图像会受到光照变化、云雾遮挡、树木生长以及季节更替的强烈干扰，模型可能会去误把这些“干扰”当做 feature，从而忽略物体本身。
- 3. 尺度不一致**：无人机飞行高度的变化会导致地面物体在图像中的尺度（Scale）发生剧烈波动。

II. Baselines & Limitations

我准备选用 **University-1652** 作为 Benchmark Dataset，因为它包含了多高度、多角度的无人机数据，是目前该领域的“黄金标准”，约等于 ImageNet / CoCo 在图像识别 / 目标检测领域的地位。

主流 Baseline 模型及其短板：

1. ResNet-50 + Triplet Loss (经典 CNN)

ResNet-50 是经典的 CNN 结构，CNN 的感受野有限，难以捕捉长距离的上下文信息；且对旋转非常敏感，一旦无人机视角旋转，特征匹配准确率会断崖式下跌。ResNet-50 和 VGG-16 这种基于 CNN 的模型放在实验里就是用来踩的（手动狗头），因为我感觉我没有自信能刷到 SOTA，FSRA（下文）的性能已经很好了，我准备换一个角度去叙述。

2. FSRA (CVPR 2022 / SOTA 代表)

以 FSRA 为代表的 SOTA 模型基本都是引入 Vision Transformer / Swin Transformer 并设计了特定的切块对齐模块，Transformer 的注意力机制约等于开天眼，在这种任务上全局感受野是随便吊打 CNN 的局部感受野的。但是目前主流的 SOTA 模型大致有以下几个共同的待改进之处：

- 1. 对几何旋转缺乏本质解决方案**：Swin Transformer 基于矩形窗口注意力机制，旋转会导致特征在窗

口间错位，FSRA 给出的解决方案主要靠数据量（堆有标签数据的数量 + 数据增强）硬抗，缺乏几何层面的矫正。

2. **对环境噪声过拟合**：模型往往倾向于学习树木、车辆等“干扰项”的纹理，而不是建筑物的本质结构。一旦遇到遮挡（如树遮住楼），检索不仅失效，甚至会匹配到错误的地点。
3. **难例挖掘不足**：在密集城区，许多楼房外观极其相似（Hard Negatives）。现在采用的 Triplet Loss 在训练后期往往忽略了这些细微差别，导致模型“分不清双胞胎”。

III. Proposed Methodology

针对上面提到的主流 SOTA 存在的问题，我计划在 Swin Transformer 架构基础上，引入三个核心模块，从几何、特征、策略三个维度进行改进。

1. 极坐标变换模块 (Polar-Transform Module)：解决“旋转”痛点 —— 将旋转转化为平移

卷积网络（CNN）和 Swin Transformer 都有一个共同特性：**对“平移”很鲁棒，但对“旋转”很敏感**。在笛卡尔坐标系 (x, y) 中，图像旋转是一个复杂的非线性变换，网络很难对齐特征。但是，在极坐标系 (ρ, θ) 中，原图的**“中心旋转”在数学上等价于图像沿角度轴（ θ -axis）的“线性平移”**。具体操作上可以在 Swin Transformer Backbone 之前插入一个可微的极坐标采样层。将正方形的输入图像通过双线性插值映射到极坐标网格上。如果我的猜想是正确的话，无论无人机怎么旋转，经过该模块处理后的图像，仅仅是发生了左右平移。这恰好迎合了 Swin Transformer 擅长处理平移的特性，从而大大减轻了网络学习旋转不变性的负担。

2. 自监督对比学习分支 (Self-Supervised Contrastive Regularization)：强迫模型透过干扰看本质

现有的训练思路容易造成**模型对环境噪声过拟合**，模型看上去更喜欢学习树木车辆之类的纹理，但是这类特征在实际中是极易变化的。为了防止模型“死记硬背”树木或汽车的位置，可以引入对比学习（Contrastive Learning）的思想作为辅助任务。具体实现上可以在训练时构建一个**孪生分支**：分支的**原始流**就是输入正常的无人机图像 I ；而**干扰流**则是输入经过强数据增强的图像 I' （例如：随机挖掉一块像素模拟遮挡、高斯模糊模拟对焦失败、剧烈颜色抖动模拟光照变化）。

约束目标上可以强制网络提取出的特征 $f(I)$ 和 $f(I')$ 必须高度相似（即 Minimize Cosine Similarity Loss，或者采用 InfoNCE-Loss 也可以，不知道 Cosine Similarity Loss 和 InfoNCE Loss 哪个好，这个估计要实测测试一下了）。这相当于给模型施加了一个正则化约束——“不管外界环境怎么变，你必须认出这是同一个房子”。这迫使网络忽略表层的环境噪声，专注于提取建筑物本身的结构特征。

3. 在线难例挖掘策略 (Online Hard-Mining Strategy): 动态攻克难样本

传统的 Triplet Loss 是随机采样的，这会导致训练后期充满了“简单样本”（一眼就能分清的图），导致梯度消失，网络学不到细微特征，模型最终难以收敛。所以我觉得可以摒弃原始的“全部数据加载后随机开奖”离线采样策略，改为在线 (In-Batch) 动态加权。在每一个训练 Batch（如 32 张图）内部，实时计算所有样本对的距离矩阵。

接下来进行**自动筛选**，也就是找出那些“明明不是同一个地方，但距离却很近”的负样本 (Hard Negatives)，以及“明明是同一个地方，距离却很远”的正样本 (Hard Positives)。在计算 Loss 时，大幅**提高这些难样本的权重**。个人感觉这种操作就像让学生专门复习“错题本”。网络在训练后期会将注意力全部集中在区分那些长得非常像的建筑物上，显著提升 Top-1 命中率（因为要刷分，只能面向数据集编程了，手动狗头）。

IV. Model Architecture Design

构建一个端到端的双流网络，具体流程如下：

1. 预处理层：Polar Spatial Transform

- **Input:** $256 \times 256 \times 3$ (Drone Image).
- **Action:** 执行笛卡尔坐标到极坐标的映射。
- **Output:** $256 \times 256 \times 3$ (Polar Image). 旋转变换为平移。

2. 特征提取主干：Swin Transformer (Tiny)

- **Backbone:** 选用 Swin-T 以平衡性能与显存。
- **Input:** Polar Image.
- **Output:** 特征图 $F \in \mathbb{R}^{768 \times 8 \times 8}$ 。

3. 特征聚合层：GeM Pooling

使用广义平均池化 (Generalized Mean Pooling) 替代普通平均池化（这个操作我暂时不知道原理，但是貌似目前主流的方案都会采取 GeM Pooling，我不是很想在这个上面做改进设计，所以直接照抄过来吧），以自适应地聚焦于显著特征区域（如建筑边缘），忽略背景（如天空、草地），随后通过全连接层 (FC) 降维至 512 维特征向量。

4. 多任务损失函数 (Loss Function)

我把模型的总损失函数 L_{total} 设计成由三部分组成，分别承担不同的指导角色，共同优化模型性能：

$$L_{total} = \underbrace{L_{triplet}}_{1. \text{ 拉近距离}} + \underbrace{L_{id}}_{2. \text{ 快速收敛}} + \lambda \cdot \underbrace{L_{contrastive}}_{3. \text{ 抗干扰训练}}$$

1. $L_{triplet}$: 三元组损失 (Triplet Loss with Hard Mining)

这是图像检索任务中最核心的 Loss，负责学习特征空间的距离度量。每次训练时我们都会构建一个“三元组” (A, P, N) ：其中 **Anchor (A)** 是锚点图（如：无人机视角的建筑）；**Positive (P)** 是正样本（即同一地点的卫星图）；**Negative (N)**：负样本（即不同地点的卫星图）。训练的约束目标是正样本要比负样本近出一个安全距离，即满足：

$$\text{Distance}(A, P) + \text{Margin} < \text{Distance}(A, N)$$

式中，Margin 是一个安全距离（比如 0.3），意思是正样本不仅要比负样本近，还要近出一段安全距离以防止模棱两可。

改进点 (Hard Mining), 对应 idea3：不同于惯常操作，我不打算使用随机采样，而是采用 **在线难例挖掘 (Online Hard Mining)** 策略，专门去刻意挑选那些“最难区分”的样本（例如：外观极其相似但不是同一栋楼的负样本），以解决随机采样带来的梯度消失问题，迫使模型关注细微的结构差异，提升精细分辨力。

2. L_{id} : 分类损失 (Cross-Entropy / ID Loss)

这是该领域的 **Standard Practice**，作为这类问题（包括行人重识别问题）训练的“基本操作”，我觉得没有必要去在这个上面做修改工作了，直接借鉴过来。大概原理就是将地理定位问题转化为一个标准的 N 分类问题（ N 为地点总数，如 701 类）。使用 Softmax + Cross-Entropy Loss 计算。其作用如同 **稳定剂**，因为单纯的 Triplet Loss 难以收敛，分类 Loss 能提供强梯度的监督信号，以帮助模型在训练初期快速学到地点的粗粒度特征。

3. $L_{contrastive}$: 自监督对比损失 (Contrastive Loss)

这是这个设计的 **创新点之一**，对应 **Idea 2**。输入原图 I 和经过强数据增强（变色、模糊、遮挡、.etc）的增强图 I' 。在总 Loss 里加入 Contrastive Loss 从而强制要求模型提取的特征 $f(I)$ 和 $f(I')$ 保持高度一致（即 Minimize Cosine Similarity Loss），同时与其他图像的特征推开。相当于一种 **正则化约束**，从而显著提升模型在恶劣环境下的鲁棒性。

4. 关于平衡系数 λ

λ 是用于调节主任务（检索）与辅助任务（对比正则化）之间的权重，其取值通常设为 $0.1 \sim 1.0$ （例如 0.5）。以确保模型以地理定位精度为主，同时兼顾抗干扰能力的学习。

具体的代码逻辑结构（伪代码说明，只是大致的程序框架设计）：

1. DataLoader

一般的模型对于原始的、从文件里直接读到的数据，处理后得到的是 $(img, label)$ 二元对，其中 **label** 是训练的标签，这个数据集上是 **地点 ID**。现在为了实现 idea2 要让 DataLoader 吐出三样东西： $(img_orig, img_aug, label)$ ，其中 **img_orig** 和 **img_aug** 分别是原始的无人机图像（只做了 Resize 和 Normalize 这类基本操作）以及数据增强后的无人机图像（在 **img_orig** 之上又叠加了 RandomGrayscale, GaussianBlur, RandomErasing 之类的破坏性增强）。

2. 输入拼接 (Concatenation)

可以不跑两次 `forward` 过程，直接把原图和增强图拼成同一个 Batch 一次送进去。假设 `Batch Size = N`，则 `x_orig.shape = x_aug.shape = [N,3,256,256]`，执行拼接操作 `x_cat = torch.cat([x_orig,x_aug],dim=0)`，得到拼接后的 `x_cat.shape = [2N,3,256,256]`。

3. 前向传播 (Forward Pass)

把这个 `2N` 大小的 Batch 送入 Swin Transformer (下面是伪代码):

```
features_cat = model.backbone(x_cat) # 输出 [2N, 768]
embeddings_cat = model.head(features_cat) # 输出 [2N, 512] (最终特征)
# 这里的 model.backbone 是 Swin Transformer, model.head 是分类头
```

4. 特征拆分

经过网络之后再把这 `2N` 个特征拆开，变回原来的两组:

```
f_orig, f_aug = torch.split(embeddings_cat, N, dim=0)
# f_orig: [N, 512] → 用于和卫星图算 Triplet Loss
# f_aug: [N, 512] → 专门用于和 f_orig 算对比 Loss
```

5. 反向传播 (设计 Loss)

上面提到了总的 Loss 分成三个部分，在训练的一个 Step 里，Loss 计算的伪代码如下:

```
def compute_total_loss(f_orig, f_aug):
    # 1. 算主任务 Loss (Triplet)
    # 用 f_orig (无人机) 和 f_satellite (卫星) 算 Triplet
    loss_triplet, loss_id = compute_retrieval_loss(f_orig,
    f_satellite, labels)
    # 2. 算辅助正则化 Loss (Contrastive)
    # 用 f_orig 和 f_aug 算
    loss_contrast = compute_contrastive_loss(f_orig, f_aug)
    # 3. 总 Loss
    total_loss = loss_triplet + loss_id + 0.5 * loss_contrast
    # 0.5 是权重系数 lambda, 是我瞎蒙的, 这个也要做实验调参
    return total_loss
```

关于 `compute_contrastive_loss(f_orig, f_aug)` 函数，上面说到有两种计算方法:


```

# 第一种，直接用余弦相似度来度量
def compute_contrastive_loss(f_orig, f_aug):
    # 解释：余弦相似度最大是1，Loss 最小是0。
    loss_contrast = 1 - torch.nn.functional.cosine_similarity(f_orig,
f_aug, dim=1).mean()
    return loss_contrast

# 第二种，用 InfoNCE 来度量
def compute_contrastive_loss(f_orig, f_aug):
    # 假设已有 f_orig 和 f_aug，都做了归一化 (normalize)
    logits = torch.matmul(f_orig, f_aug.T) / temperature # [N, N] 的矩
    阵
    # 对角线是正样本 (自己 vs 自己的增强版)
    labels = torch.arange(N).to(device)
    # 就像做一个 N 分类的 CrossEntropy
    loss_contrast = torch.nn.CrossEntropyLoss()(logits, labels)
    return loss_contrast

```

两个我也不知道谁更好使，只能大力出奇迹都试一下了。

V. Experimental Design

1. 数据集与评估指标

- **数据集**：University-1652（必须，貌似也只有这个了），SUES-200（备选，用于验证泛化性）。
- **评价指标**：**R@1 (Recall at Top 1)** 是必选的，是最重要的指标，代表首次匹配即成功的概率。除此之外还可以有**AP (Average Precision)**，用于反映检索结果的综合排序质量，或者 Recall at Top-K（视情况而定）

2. 主实验

设计实验将与以下模型进行对比，目标是在 R@1 上超越它们：首先就是前文提到的传统的基于 CNN 的方法 **VGG16 / ResNet-50**，目的是为了证明 Transformer 架构的优势（这个属实是没的选了，只能拉出 CNN 来踩一捧一了）。其次是之前的 SOTA **LPN (Local Pattern Network)**，证明我们方法的整体性优势。再一个就是 **FSRA (SOTA)**了，FSRA是重点对比对象。即使 R@1 提升不大（甚至我感觉大概率是刷不过 FSRA 的），也要强调在恶劣环境下更强（只能这样强凑一下 novelty 了，不然我也不知道咋操作）。

3. 消融实验 (Ablation Study)

这是证明 Idea 有效性的关键环节，也没有什么技术含量，就是逐步添加模块，观察 R@1（还可以有其他指标，工作量取决于有多少人来做实验以及他们的工作效率如何）的变化：

实验组	极坐标变换 (Polar)	对比正则化 (Contrast)	难例挖掘 (Hard Mining)	结论说明
Baseline	×	×	×	原始 Swin Transformer 性能
Exp 1	√	×	×	证明几何对齐有效，解决旋转问题
Exp 2	√	√	×	证明鲁棒性增强
Exp 3 (Full)	√	√	√	证明难例挖掘提升了精细分辨力

4. 鲁棒性专项测试 (Robustness Analysis)

还是那句话，我感觉大概率是刷不到 SOTA 的水平（因为没有什么其他的标准了，只有 University-1652，在这上面很难去 pk 得过 FSRA）。当然，为了向审核“凸显我们的优势”，我觉得除了看总分还要专门做一个图表，以证明我们的“鲁棒性”（这一点是 Daiming 团队没有做的点）：可以人为对测试集施加干扰（如：降低 50% 亮度 或 随机遮挡 20% 画面 或其它引入噪声的手段），然后对比 Ours 和 FSRA 在干扰下的性能下降幅度。预期结果：（这相当于我们在面向任务编程强凑了，但是只能在这个方向上去刷了）因为 FSRA 的设计里没有做特殊的数据增广 + 采用自监督对比学习的双流孪生网络训练方式（idea 2 里面的操作），所以此时 FSRA 分数可能会暴跌，而我们的模型因为有“对比正则化”，分数下降应该更平缓，证明我们的模型更“皮实”。